

## **Advanced Data Management**

### **Project Report**

City Mobility Platform — Data Management System

Academic Year 2025/2026

University of Milano-Bicocca

**Instructors:** Gloria Lopiano & Andrea Campagner

**Student Name:** Md Rifatul Haque

**Matriculation Number:** 947512

## Part 1: Relational vs Document Model

In this part, we build the same database using two different approaches: a traditional relational database (PostgreSQL) and a document database (MongoDB). We then compare how well each one handles our four queries at different data sizes.

### 1.1 Data Modeling

#### Relational Schema — PostgreSQL

In PostgreSQL, data is organized into four separate tables that are linked together using foreign keys (like references between tables):

- users — stores each user's name, surname, birthdate, and country
- stations — stores each station's name, city, and maximum vehicle capacity
- trips — stores each trip, linking a user to a start station and end station, with start/end times and cost
- events — stores each event that happened during a trip (GPS, ERROR, BATTERY, or DELAY)

Indexes were added on all linking columns (foreign keys) and on the event type column to make queries faster.

#### Document Schema — MongoDB

In MongoDB, data is stored as documents (similar to JSON files). We use three collections:

- users collection — one document per user
- stations collection — one document per station
- trips collection — one document per trip, with all events stored inside the trip document (embedded)

The most important design decision is that events are stored inside their trip document. This is called embedding. Users and stations are stored separately and referenced by ID. This is called referencing.

#### Why Embedding vs Referencing?

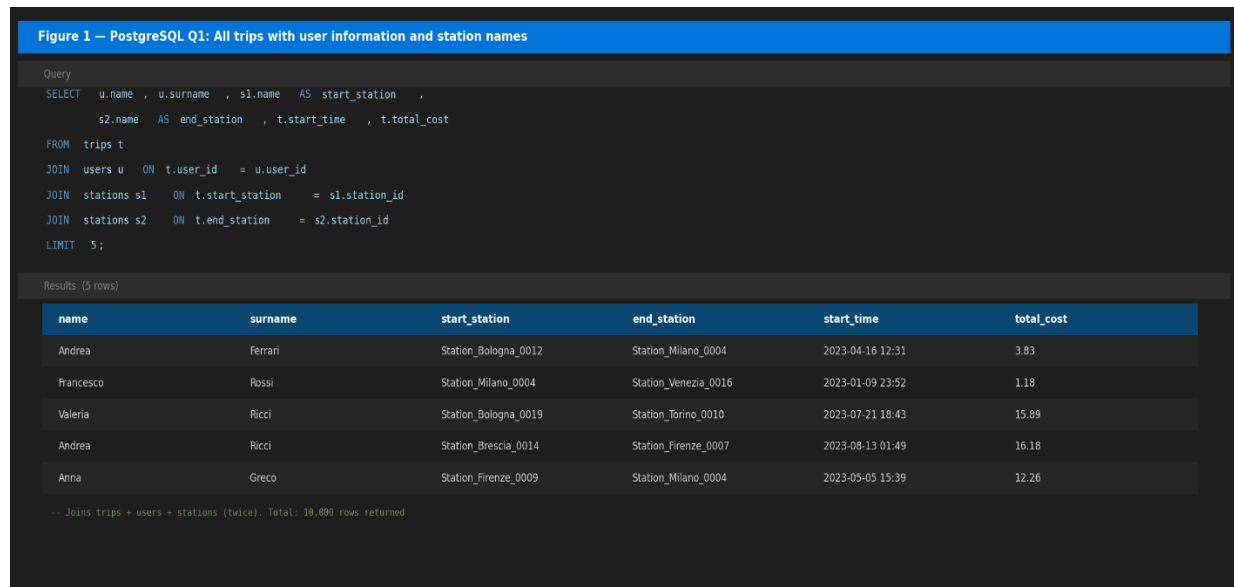
| What     | Decision              | Why                                                                                                                     |
|----------|-----------------------|-------------------------------------------------------------------------------------------------------------------------|
| Events   | EMBEDDED inside trips | Events only make sense as part of a trip. Storing them inside the trip avoids slow joins.                               |
| Users    | REFERENCED (separate) | One user can have many trips. If we embedded user info in every trip, we would repeat the same data thousands of times. |
| Stations | REFERENCED (separate) | Same reason — only ~20 stations serve thousands of trips. Repeating station data would waste space.                     |

## 1.2 Query Implementation

We implemented the same four queries in both PostgreSQL and MongoDB, then measured how long each one takes at different data sizes.

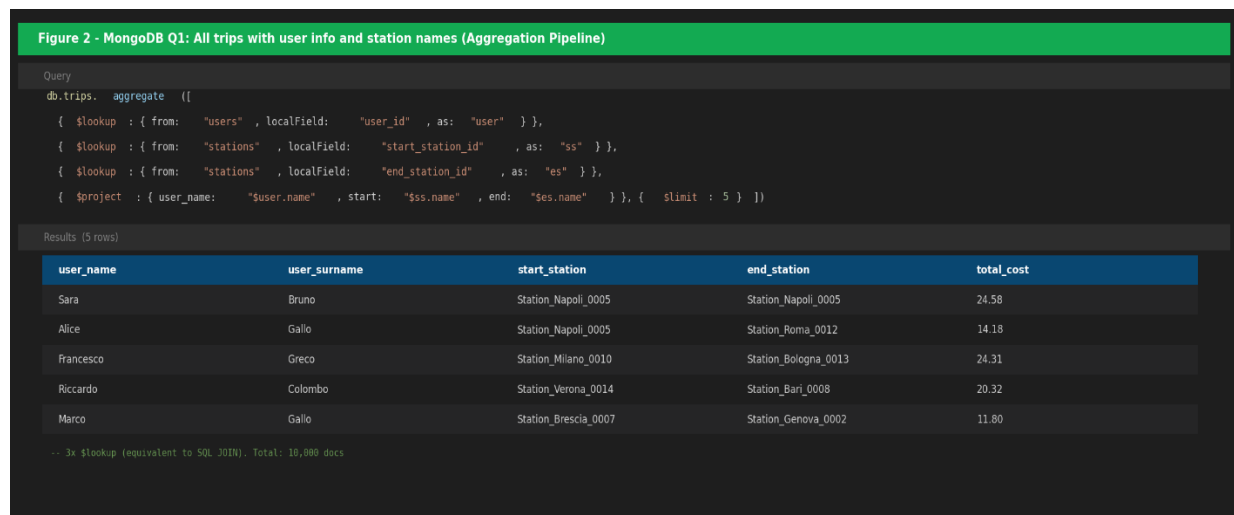
### Query 1 — Show all trips with user details and station names

**PostgreSQL Approach:** We join three tables together — trips, users, and stations (twice for start and end). Because all linking columns are indexed, this is fast.



*Figure 1: PostgreSQL - Q1: All trips with user information and station names*

**MongoDB Approach:** We use three \$lookup operations (similar to joins). This is slower than PostgreSQL because MongoDB has to look up referenced documents separately at query time.



*Figure 2: MongoDB - Q1: All trips with user information and station names*

### Query 2 — Show each user with their trip count and average trip duration

**PostgreSQL Approach:** Group all trips by user and calculate COUNT and AVG. Very efficient with indexed columns.

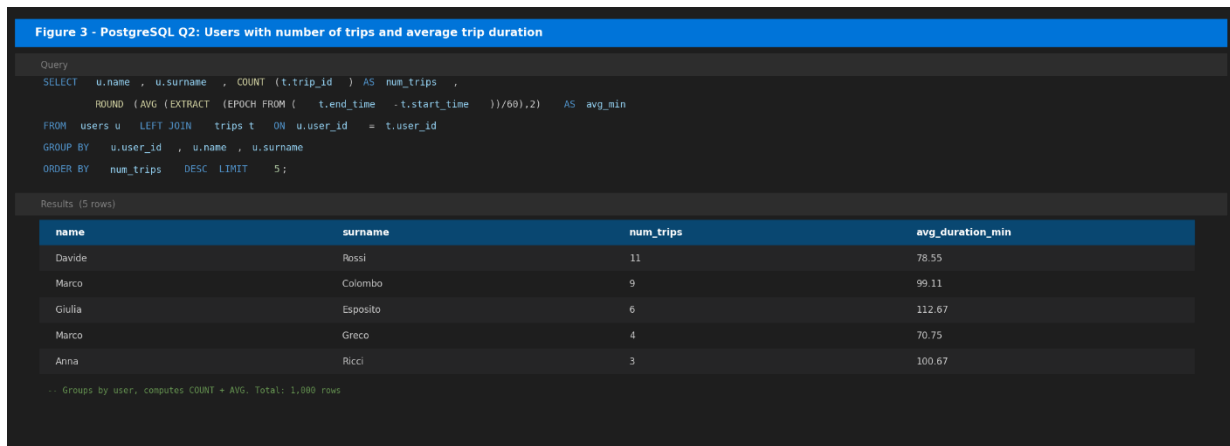


Figure 3: PostgreSQL - Q2: Users with number of trips and average trip duration

**MongoDB Approach:** Look up each user's trips and compute statistics. Slower when there are many users (50k) because of the lookup overhead.

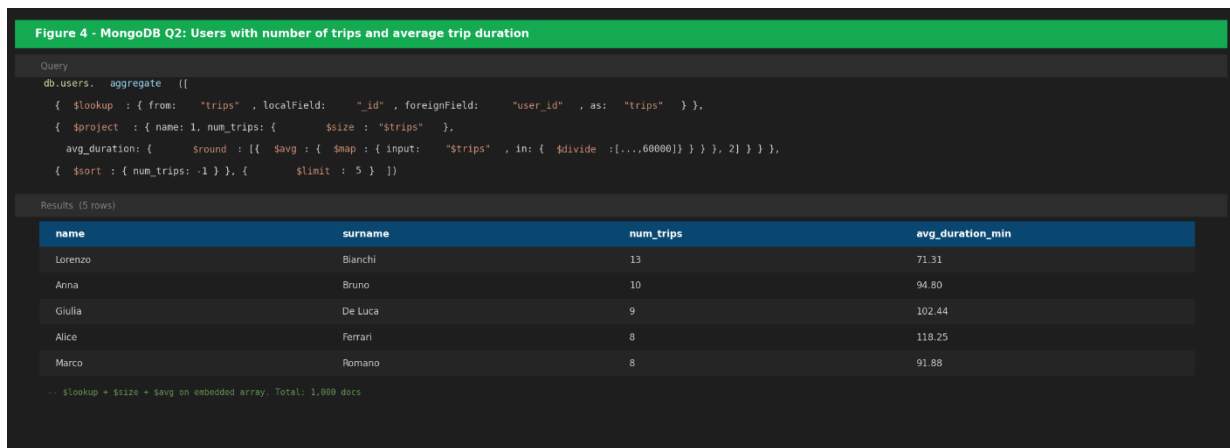


Figure 4: MongoDB - Q2: Users with number of trips and average trip duration

## Query 3 — Show each station with how many trips start and end there

**PostgreSQL Approach:** Join the trips table twice (once for start, once for end). This creates a very large intermediate result, which makes it extremely slow at large scales.

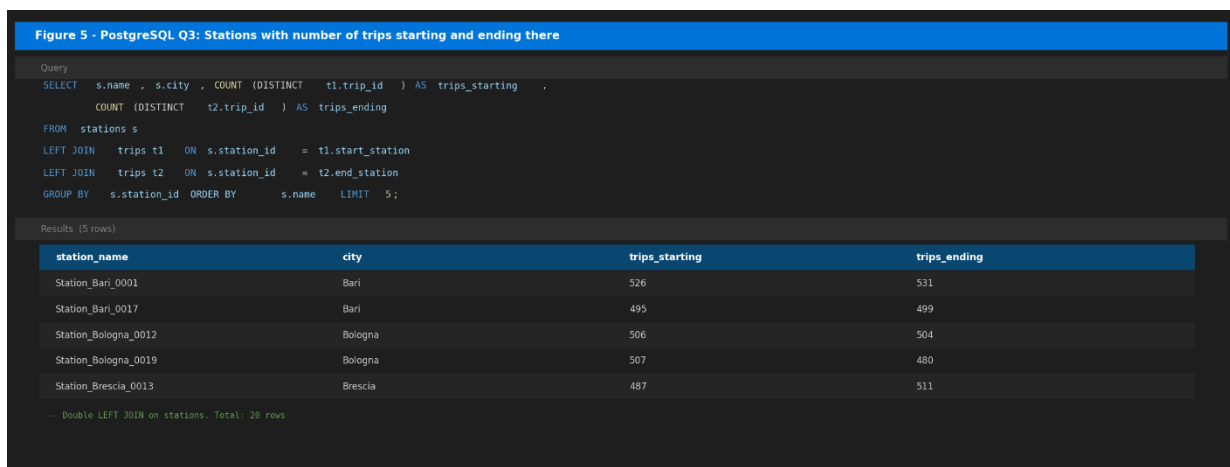


Figure 5: PostgreSQL - Q3: Stations with trips starting and ending there

**MongoDB Approach:** Use \$facet to count start trips and end trips separately and efficiently. This is much, much faster than PostgreSQL at large scales.

## Query 4 — Find all trips that have at least one ERROR event

**PostgreSQL Approach:** Use an EXISTS check on the events table — stops as soon as it finds one matching event, so it is efficient.

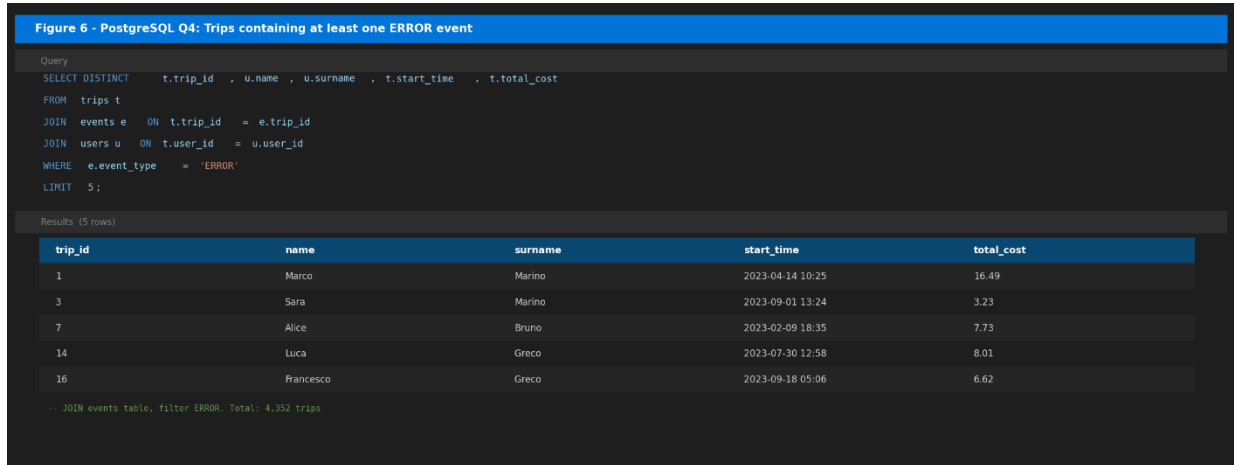


Figure 6: PostgreSQL - Q4: Trips containing at least one ERROR event

**MongoDB Approach:** Filter directly on the embedded events array. Because events are inside the trip document, no separate table lookup is needed.

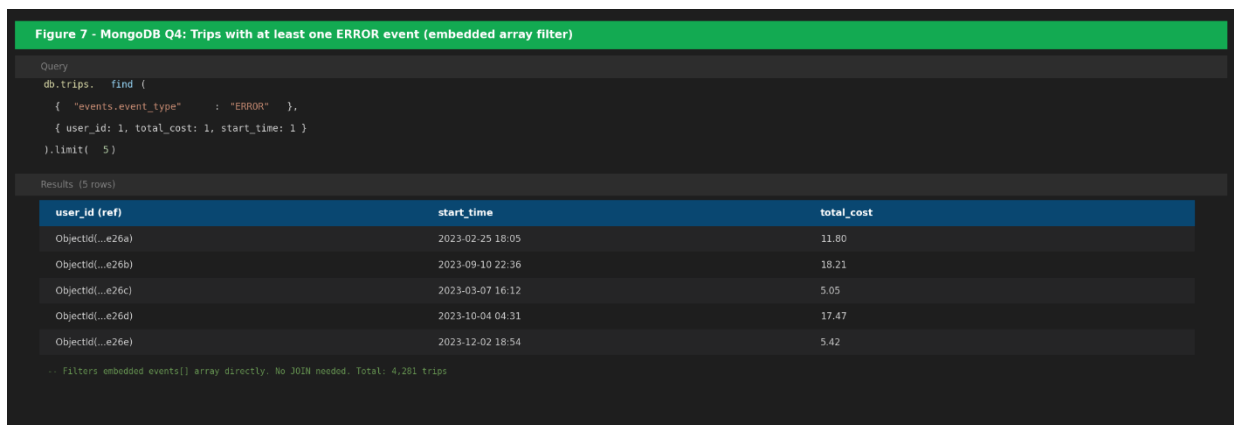


Figure 7: MongoDB - Q4: Trips containing at least one ERROR event

## Benchmark Results — All 36 Scale Combinations

We tested every combination of: 1k/10k/50k users × 10k/50k/100k trips × 0/2/5/10 events per trip. Times shown in seconds (lower = faster):

| DB | Users | Trips | Events | Q1 (s) | Q2 (s) | Q3 (s) | Q4 (s) |
|----|-------|-------|--------|--------|--------|--------|--------|
| PG | 1k    | 10k   | 0      | 0.20   | 0.02   | 3.92   | 0.02   |
| MG | 1k    | 10k   | 0      | 2.02   | 0.06   | 0.02   | 0.01   |
| PG | 1k    | 10k   | 2      | 0.50   | 0.01   | 3.26   | 0.16   |
| MG | 1k    | 10k   | 2      | 2.19   | 0.04   | 0.02   | 0.07   |

| DB | Users | Trips | Events | Q1 (s) | Q2 (s) | Q3 (s) | Q4 (s) |
|----|-------|-------|--------|--------|--------|--------|--------|
| PG | 1k    | 10k   | 5      | 0.58   | 0.02   | 11.71  | 0.64   |
| MG | 1k    | 10k   | 5      | 2.51   | 0.04   | 0.02   | 0.14   |
| PG | 1k    | 10k   | 10     | 0.78   | 0.04   | 5.18   | 0.45   |
| MG | 1k    | 10k   | 10     | 3.58   | 0.14   | 0.04   | 0.16   |
| PG | 1k    | 50k   | 0      | 3.08   | 0.03   | 146.71 | 0.01   |
| MG | 1k    | 50k   | 0      | 14.37  | 0.38   | 0.16   | 0.02   |
| PG | 1k    | 50k   | 2      | 2.94   | 0.07   | 206.13 | 1.08   |
| MG | 1k    | 50k   | 2      | 17.30  | 0.47   | 0.26   | 0.55   |
| PG | 1k    | 50k   | 5      | 5.26   | 0.08   | 228.59 | 2.10   |
| MG | 1k    | 50k   | 5      | 23.36  | 0.55   | 0.15   | 0.79   |
| PG | 1k    | 50k   | 10     | 3.35   | 0.11   | 214.55 | 2.32   |
| MG | 1k    | 50k   | 10     | 15.28  | 0.34   | 0.23   | 0.59   |
| PG | 1k    | 100k  | 0      | 5.21   | 0.12   | 957.68 | 0.01   |
| MG | 1k    | 100k  | 0      | 21.67  | 0.49   | 0.30   | 0.02   |
| PG | 1k    | 100k  | 2      | 6.13   | 0.09   | 721.25 | 1.59   |
| MG | 1k    | 100k  | 2      | 29.55  | 0.99   | 0.42   | 0.68   |
| PG | 1k    | 100k  | 5      | 5.91   | 0.06   | 770.31 | 1.33   |
| MG | 1k    | 100k  | 5      | 21.98  | 0.86   | 0.23   | 0.95   |
| PG | 1k    | 100k  | 10     | 2.18   | 0.05   | 751.89 | 1.34   |
| MG | 1k    | 100k  | 10     | 32.39  | 1.04   | 0.39   | 1.35   |
| PG | 10k   | 10k   | 0      | 0.23   | 0.04   | 0.19   | 0.09   |
| MG | 10k   | 10k   | 0      | 2.55   | 0.25   | 0.05   | 0.02   |
| PG | 10k   | 10k   | 2      | 0.42   | 0.06   | 0.30   | 0.10   |
| MG | 10k   | 10k   | 2      | 3.63   | 0.37   | 0.08   | 0.10   |
| PG | 10k   | 10k   | 5      | 0.41   | 0.04   | 0.29   | 0.37   |
| MG | 10k   | 10k   | 5      | 4.58   | 0.49   | 0.04   | 0.19   |
| PG | 10k   | 10k   | 10     | 0.35   | 0.07   | 0.21   | 0.19   |
| MG | 10k   | 10k   | 10     | 3.50   | 0.52   | 0.11   | 0.20   |
| PG | 10k   | 50k   | 0      | 1.65   | 0.08   | 19.02  | 0.01   |
| MG | 10k   | 50k   | 0      | 9.89   | 0.45   | 0.15   | 0.02   |
| PG | 10k   | 50k   | 2      | 1.43   | 0.15   | 20.73  | 0.51   |
| MG | 10k   | 50k   | 2      | 16.44  | 0.56   | 0.17   | 0.28   |
| PG | 10k   | 50k   | 5      | 1.29   | 0.05   | 11.89  | 0.57   |

| DB | Users | Trips | Events | Q1 (s) | Q2 (s) | Q3 (s) | Q4 (s) |
|----|-------|-------|--------|--------|--------|--------|--------|
| MG | 10k   | 50k   | 5      | 9.76   | 0.68   | 0.12   | 0.48   |
| PG | 10k   | 50k   | 10     | 1.10   | 0.08   | 11.61  | 0.72   |
| MG | 10k   | 50k   | 10     | 9.58   | 0.49   | 0.09   | 0.42   |
| PG | 10k   | 100k  | 0      | 1.91   | 0.08   | 60.55  | 0.00   |
| MG | 10k   | 100k  | 0      | 27.83  | 0.85   | 0.32   | 0.05   |
| PG | 10k   | 100k  | 2      | 2.03   | 0.11   | 51.28  | 0.68   |
| MG | 10k   | 100k  | 2      | 25.27  | 0.77   | 0.32   | 0.71   |
| PG | 10k   | 100k  | 5      | 2.15   | 0.09   | 55.11  | 1.13   |
| MG | 10k   | 100k  | 5      | 25.29  | 0.94   | 0.29   | 1.02   |
| PG | 10k   | 100k  | 10     | 2.22   | 0.24   | 58.63  | 1.34   |
| MG | 10k   | 100k  | 10     | 20.86  | 0.76   | 0.27   | 0.93   |
| PG | 50k   | 10k   | 0      | 0.31   | 0.23   | 0.08   | 0.07   |
| MG | 50k   | 10k   | 0      | 2.04   | 0.89   | 0.12   | 0.02   |
| PG | 50k   | 10k   | 2      | 0.32   | 0.29   | 0.04   | 0.06   |
| MG | 50k   | 10k   | 2      | 2.31   | 1.12   | 0.04   | 0.05   |
| PG | 50k   | 10k   | 5      | 0.26   | 0.14   | 0.04   | 0.11   |
| MG | 50k   | 10k   | 5      | 2.24   | 1.39   | 0.13   | 0.12   |
| PG | 50k   | 10k   | 10     | 0.30   | 0.12   | 0.04   | 0.16   |
| MG | 50k   | 10k   | 10     | 2.34   | 1.15   | 0.03   | 0.11   |
| PG | 50k   | 50k   | 0      | 1.12   | 0.14   | 0.71   | 0.00   |
| MG | 50k   | 50k   | 0      | 11.95  | 1.66   | 0.18   | 0.03   |
| PG | 50k   | 50k   | 2      | 1.16   | 0.17   | 0.79   | 0.29   |
| MG | 50k   | 50k   | 2      | 10.08  | 1.34   | 0.15   | 0.28   |
| PG | 50k   | 50k   | 5      | 1.13   | 0.16   | 0.63   | 0.52   |
| MG | 50k   | 50k   | 5      | 12.41  | 1.88   | 0.18   | 0.50   |
| PG | 50k   | 50k   | 10     | 1.12   | 0.17   | 0.61   | 0.61   |
| MG | 50k   | 50k   | 10     | 12.69  | 1.65   | 0.17   | 0.52   |
| PG | 50k   | 100k  | 0      | 2.28   | 0.34   | 7.39   | 0.00   |
| MG | 50k   | 100k  | 0      | 27.18  | 2.11   | 0.34   | 0.02   |
| PG | 50k   | 100k  | 2      | 2.21   | 0.19   | 5.42   | 0.63   |
| MG | 50k   | 100k  | 2      | 22.61  | 2.01   | 0.41   | 0.42   |
| PG | 50k   | 100k  | 5      | 2.18   | 0.15   | 4.47   | 0.98   |
| MG | 50k   | 100k  | 5      | 22.57  | 2.14   | 0.56   | 1.07   |

| DB | Users | Trips | Events | Q1 (s) | Q2 (s) | Q3 (s) | Q4 (s) |
|----|-------|-------|--------|--------|--------|--------|--------|
| PG | 50k   | 100k  | 10     | 2.09   | 0.18   | 4.93   | 1.27   |
| MG | 50k   | 100k  | 10     | 20.36  | 1.72   | 0.29   | 0.93   |

## What the Results Tell Us

**Query 1 (trips with user and station info):** PostgreSQL is clearly faster. At 50k users and 100k trips, PostgreSQL takes 2.09 seconds while MongoDB takes 20.36 seconds. SQL joins on indexed columns are very efficient.

**Query 2 (user statistics):** PostgreSQL wins again, especially when there are many users. At 50k users and 100k trips, PostgreSQL takes 0.18 seconds vs MongoDB's 1.72 seconds. MongoDB slows down because it must look up each user's trips separately.

**Query 3 (station trip counts):** MongoDB wins dramatically. The most extreme example: at 1k users and 100k trips, PostgreSQL takes 957 seconds (about 16 minutes!) while MongoDB takes only 0.30 seconds. This is because PostgreSQL's double join creates an enormous temporary result that is very slow to process.

**Query 4 (trips with ERROR events):** Results depend on event density. With no events, both are very fast. With 10 events per trip, both slow down similarly. MongoDB's embedded events give a slight advantage because no separate table lookup is needed.

## 1.3 Schema Evolution — Adding a New Field

The professor asked: what happens if we need to add a 'battery\_level' field (a number from 0 to 100) to all BATTERY events? Here is how each database handles this change:

### PostgreSQL — How to add the field

We need to run this command: `ALTER TABLE events ADD COLUMN battery_level INTEGER CHECK (battery_level BETWEEN 0 AND 100);`

This adds the column to every existing row with a NULL value (meaning 'no data'). For very large tables, this command can lock the table temporarily, which means no one can use it while the change is being made. The application code also needs to be updated to fill in battery\_level for BATTERY events.

### MongoDB — How to add the field

In MongoDB, we do not need to run any migration command at all. We simply start saving new BATTERY event documents with the battery\_level field included: `{ event_type: 'BATTERY', value: '85', battery_level: 85 }`. Old documents without the field remain perfectly valid. MongoDB is flexible enough to handle documents with different fields in the same collection.

| Aspect                   | PostgreSQL                 | MongoDB             |
|--------------------------|----------------------------|---------------------|
| Migration command needed | Yes — ALTER TABLE required | No — nothing needed |



| Aspect                  | PostgreSQL                                    | MongoDB                                         |
|-------------------------|-----------------------------------------------|-------------------------------------------------|
| Risk during migration   | Table lock possible (no access during change) | No risk — zero downtime                         |
| Effect on existing data | NULL added to all existing rows               | Existing documents unchanged                    |
| Validation              | CHECK constraint enforces 0-100 range         | JSON Schema validator can be updated optionally |
| Overall verdict         | Less flexible (lower evolvability)            | More flexible (higher evolvability)             |

**Conclusion:** MongoDB is much easier to evolve. Adding optional fields requires no migration, no downtime, and no risk. PostgreSQL requires a planned migration with potential service interruption.

## 1.4 Spark Implementation of Query 2

We implemented Query 2 (user trip counts and average duration) using PySpark in two ways, and compared both against the native MongoDB implementation:

- RDD approach: load trip data, map each trip to (user\_id, count=1, duration), then reduce by user to sum up counts and durations, finally divide to get the average
- DataFrame approach: create a Spark DataFrame from the trip data, use groupBy and aggregate functions (count, avg), then join with user data

| Method          | Users | Trips | Events | Time (seconds) |
|-----------------|-------|-------|--------|----------------|
| MongoDB native  | 1k    | 10k   | 2      | 0.04           |
| Spark RDD       | 1k    | 10k   | 2      | 3.82           |
| Spark DataFrame | 1k    | 10k   | 2      | 6.91           |
| MongoDB native  | 1k    | 100k  | 10     | 1.04           |
| Spark RDD       | 1k    | 100k  | 10     | 6.92           |
| Spark DataFrame | 1k    | 100k  | 10     | 14.24          |
| MongoDB native  | 50k   | 100k  | 10     | 1.72           |
| Spark RDD       | 50k   | 100k  | 10     | 7.15           |
| Spark DataFrame | 50k   | 100k  | 10     | 15.30          |

Why is MongoDB faster than Spark? Because Spark has a startup cost (it needs to initialize a computing environment) and has to serialize/deserialize data between Python and Java. For a single query on one machine, the native database engine is always faster. Spark's real advantage appears when data is distributed across many machines in a cluster — at the scale of billions of records. The RDD approach is faster than DataFrame because our data is already in Python format, so the DataFrame's SQL query planning adds unnecessary overhead.

## Part 2: Graph Model

In this part, we model the mobility data as a graph using Neo4j. A graph database is excellent for questions like 'which stations can this user reach?' because these are naturally expressed as graph traversals.

### 2.1 Graph Schema and Queries

#### Graph Design

The graph has three types of nodes and three types of edges (connections):

- **NODE: USER** — represents a registered user (stores: user\_id, name, surname, country)
- **NODE: TRIP** — represents a completed trip (stores: trip\_id, total\_cost)
- **NODE: STATION** — represents a station (stores: station\_id, name, city, capacity)
- **EDGE: PERFORMED** — connects a USER to a TRIP (meaning: this user took this trip)
- **EDGE: STARTS\_AT** — connects a TRIP to a STATION (meaning: this trip started at this station)
- **EDGE: ENDS\_AT** — connects a TRIP to a STATION (meaning: this trip ended at this station)

This structure makes it easy to answer questions by following the connections. For example, to find all stations a user visited, we simply follow: USER → PERFORMED → TRIP → STARTS\_AT/ENDS\_AT → STATION.

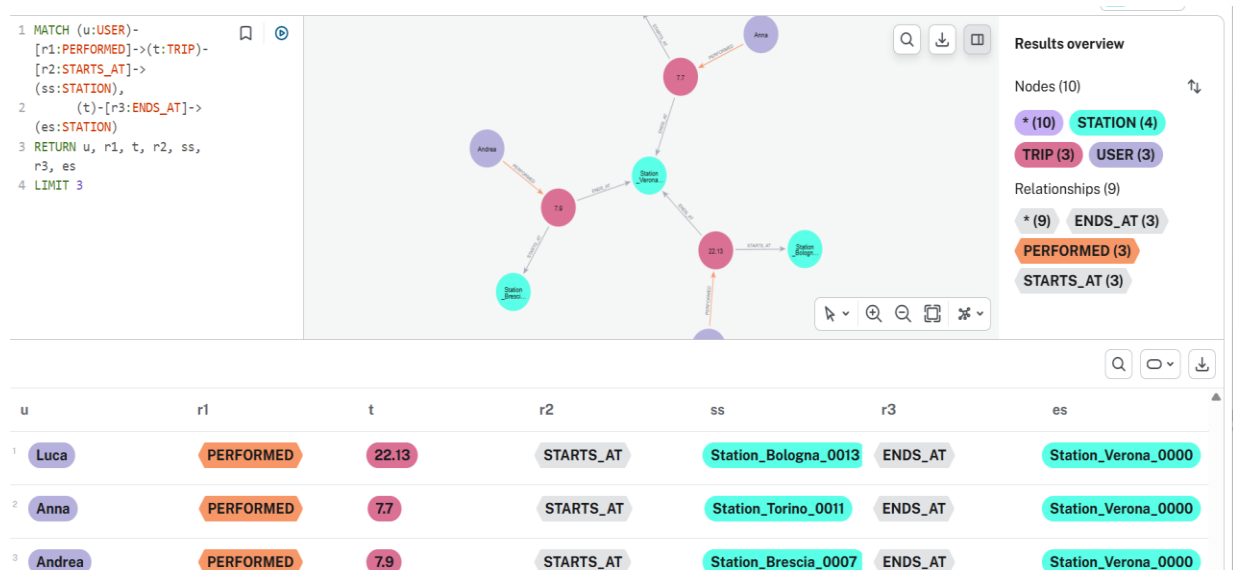


Figure 8: Neo4j - Neo4j — Graph model showing USER, TRIP and STATION nodes with PERFORMED, STARTS\_AT and ENDS\_AT relationships

#### Cypher Query 1 — All stations reachable by a given user

This query finds all stations visited by user Anna through her trips. Starting from the USER node where name = "Anna", it follows the PERFORMED relationship to reach all TRIP nodes

belonging to her, then follows STARTS\_AT and ENDS\_AT relationships to reach the connected STATION nodes. The DISTINCT keyword ensures each station appears only once even if Anna visited it multiple times.

**Result:** The query returns 19 unique stations out of 20 total stations in the database, as shown in Figure 9. This means Anna's trips have covered almost the entire station network. The one missing station was simply never used as a start or end point in her trips.

The key advantage over SQL is that no JOIN operations are needed — Neo4j traverses the relationships directly in a single pattern match. The equivalent PostgreSQL query would require two JOIN operations across three tables.

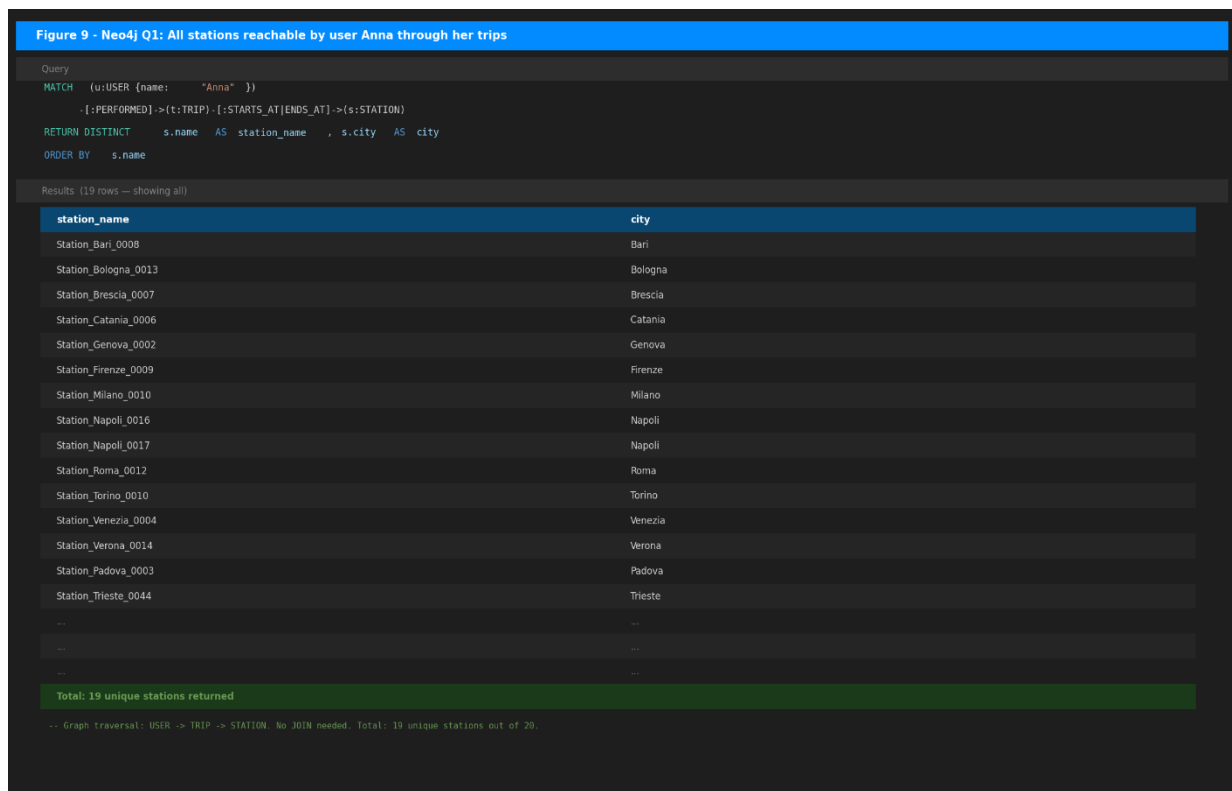


Figure 9: Neo4j - Q1: Stations reachable by user Anna via graph traversal

## Cypher Query 2 — Top 3 most important stations (by trip count)

This query identifies the three most important stations in the network based on the total number of trips connected to them. For each STATION node, it counts the number of trips that start there (STARTS\_AT relationship) and the number of trips that end there (ENDS\_AT relationship) using OPTIONAL MATCH so that stations with zero connections in one direction are still included. The two counts are added together to produce a total\_trips score, and the results are ordered from highest to lowest.

```

1 MATCH (s:STATION)
2 OPTIONAL MATCH (s)-[:STARTS_AT]-(t_out:TRIP)
3 OPTIONAL MATCH (s)-[:ENDS_AT]-(t_in:TRIP)
4 WITH s,
5     COUNT(DISTINCT t_out) AS outgoing_trips,
6     COUNT(DISTINCT t_in) AS incoming_trips
7 RETURN s.name AS station_name,
8         s.city AS city,
9         outgoing_trips,
10        incoming_trips,
11        outgoing_trips + incoming_trips AS total_trips
12 ORDER BY total_trips DESC
13 LIMIT 3

```

|   | station_name          | city     | outgoing_trips | incoming_trips | total_trips |
|---|-----------------------|----------|----------------|----------------|-------------|
| 1 | "Station_Milano_0003" | "Milano" | 552            | 552            | 1104        |
| 2 | "Station_Napoli_0016" | "Napoli" | 510            | 530            | 1040        |
| 3 | "Station_Verona_0014" | "Verona" | 510            | 525            | 1035        |

*Figure 10: Neo4j - Q2: Top 3 stations by total trip count (incoming + outgoing)*

## Neo4j Scalability Benchmark — All 36 Combinations

We tested both queries at all required scale combinations. Times in seconds:

| Users | Trips | Events | Q1 (s) | Q2 (s) |
|-------|-------|--------|--------|--------|
| 1k    | 10k   | 0      | 1.4739 | 3.958  |
| 1k    | 10k   | 2      | 0.0238 | 3.1408 |
| 1k    | 10k   | 5      | 0.0168 | 1.714  |
| 1k    | 10k   | 10     | 0.0230 | 1.4544 |
| 1k    | 50k   | 0      | 0.2435 | 51.271 |
| 1k    | 50k   | 2      | 0.0113 | 40.097 |
| 1k    | 50k   | 5      | 0.0083 | 32.645 |
| 1k    | 50k   | 10     | 0.0151 | 36.122 |
| 1k    | 100k  | 0      | 0.0148 | 146.93 |
| 1k    | 100k  | 2      | 0.0093 | 164.16 |
| 1k    | 100k  | 5      | 0.0171 | 160.33 |
| 1k    | 100k  | 10     | 0.0122 | 185.48 |
| 10k   | 10k   | 0      | 0.2906 | 0.3753 |
| 10k   | 10k   | 2      | 0.0039 | 0.1807 |
| 10k   | 10k   | 5      | 0.0039 | 0.1490 |
| 10k   | 10k   | 10     | 0.0037 | 0.1042 |
| 10k   | 50k   | 0      | 0.3228 | 3.1707 |
| 10k   | 50k   | 2      | 0.0055 | 2.8703 |
| 10k   | 50k   | 5      | 0.0039 | 3.0524 |

| Users | Trips | Events | Q1 (s) | Q2 (s) |
|-------|-------|--------|--------|--------|
| 10k   | 50k   | 10     | 0.0043 | 2.7205 |
| 10k   | 100k  | 0      | 0.0045 | 11.969 |
| 10k   | 100k  | 2      | 0.0035 | 12.483 |
| 10k   | 100k  | 5      | 0.0040 | 14.713 |
| 10k   | 100k  | 10     | 0.0069 | 13.145 |
| 50k   | 10k   | 0      | 0.1245 | 0.1908 |
| 50k   | 10k   | 2      | 0.0031 | 0.0379 |
| 50k   | 10k   | 5      | 0.0050 | 0.0464 |
| 50k   | 10k   | 10     | 0.0056 | 0.0610 |
| 50k   | 50k   | 0      | 0.1718 | 0.7907 |
| 50k   | 50k   | 2      | 0.0042 | 0.5753 |
| 50k   | 50k   | 5      | 0.0033 | 0.5261 |
| 50k   | 50k   | 10     | 0.0043 | 0.5295 |
| 50k   | 100k  | 0      | 0.0036 | 2.0173 |
| 50k   | 100k  | 2      | 0.0055 | 2.1942 |
| 50k   | 100k  | 5      | 0.0082 | 2.7405 |
| 50k   | 100k  | 10     | 0.0033 | 2.3086 |

**Key observations:** Query 1 (finding stations for a user) is extremely fast — usually under 0.02 seconds — because following one user's connections is very direct. Query 2 (counting trips per station) is slower and grows with the number of trips, because it must count relationships across all stations. At 1k users and 100k trips, Q2 takes 146-185 seconds because with only 1k users and 100k trips, each user has 100 trips on average, making the relationship counting expensive. With more users (10k or 50k), the trips are spread across more users, so each user has fewer trips and Q2 becomes much faster.

## 2.2 Spark Graph Queries

We implemented two graph algorithms using PySpark on the station subgraph. In this subgraph, each trip is an edge from its start station to its end station.

### PageRank — Finding the Most Important Stations

PageRank is an algorithm originally used by Google to rank web pages. Here, we use it to rank stations by importance. A station gets a higher PageRank if many other important stations send trips to it. We run 20 iterations with a damping factor of 0.85 (standard settings).

| Rank | Station              | PageRank Score |
|------|----------------------|----------------|
| #1   | Station_Verona_0003  | 0.052324       |
| #2   | Station_Catania_0015 | 0.052028       |
| #3   | Station_Firenze_0016 | 0.050782       |

The scores are very similar (all around 0.05) because with only 20 stations and randomly distributed trips, the network is very uniform — no station dominates the others.

### Connected Components — Is the Network Connected?

Connected Components finds groups of stations that are reachable from each other. If all stations are in one group, the entire network is connected. We use label propagation: each station starts with its own label, then repeatedly takes the smallest label from its neighbors until no more changes happen.

Result: The algorithm converged after only 2 iterations, finding 1 connected component containing all 20 stations. This means every station in the network can be reached from every other station through trips — the mobility network is fully connected.

### Spark Graph Scalability Benchmark — All 36 Combinations

| Users | Trips | Events | PageRank (s) | CC (s) | Components |
|-------|-------|--------|--------------|--------|------------|
| 1k    | 10k   | 0      | 0.042        | 0.003  | 1          |
| 1k    | 10k   | 2      | 0.029        | 0.001  | 1          |
| 1k    | 10k   | 5      | 0.017        | 0.001  | 1          |
| 1k    | 10k   | 10     | 0.032        | 0.001  | 1          |
| 1k    | 50k   | 0      | 0.224        | 0.015  | 1          |
| 1k    | 50k   | 2      | 0.146        | 0.006  | 1          |
| 1k    | 50k   | 5      | 0.102        | 0.006  | 1          |
| 1k    | 50k   | 10     | 0.249        | 0.014  | 1          |
| 1k    | 100k  | 0      | 0.224        | 0.012  | 1          |
| 1k    | 100k  | 2      | 0.345        | 0.013  | 1          |
| 1k    | 100k  | 5      | 0.166        | 0.008  | 1          |
| 1k    | 100k  | 10     | 0.294        | 0.012  | 1          |
| 10k   | 10k   | 0      | 0.020        | 0.007  | 1          |
| 10k   | 10k   | 2      | 0.026        | 0.006  | 1          |
| 10k   | 10k   | 5      | 0.030        | 0.013  | 1          |
| 10k   | 10k   | 10     | 0.029        | 0.016  | 1          |
| 10k   | 50k   | 0      | 0.240        | 0.040  | 1          |

| Users | Trips | Events | PageRank (s) | CC (s) | Components |
|-------|-------|--------|--------------|--------|------------|
| 10k   | 50k   | 2      | 0.225        | 0.022  | 1          |
| 10k   | 50k   | 5      | 0.275        | 0.051  | 1          |
| 10k   | 50k   | 10     | 0.117        | 0.014  | 1          |
| 10k   | 100k  | 0      | 0.531        | 0.069  | 1          |
| 10k   | 100k  | 2      | 0.875        | 0.087  | 1          |
| 10k   | 100k  | 5      | 0.505        | 0.053  | 1          |
| 10k   | 100k  | 10     | 0.559        | 0.053  | 1          |
| 50k   | 10k   | 0      | 0.064        | 0.014  | 1          |
| 50k   | 10k   | 2      | 0.041        | 0.015  | 1          |
| 50k   | 10k   | 5      | 0.055        | 0.021  | 1          |
| 50k   | 10k   | 10     | 0.084        | 0.059  | 1          |
| 50k   | 50k   | 0      | 0.376        | 0.299  | 1          |
| 50k   | 50k   | 2      | 0.462        | 0.140  | 1          |
| 50k   | 50k   | 5      | 0.264        | 0.114  | 1          |
| 50k   | 50k   | 10     | 0.300        | 0.120  | 1          |
| 50k   | 100k  | 0      | 0.627        | 0.259  | 1          |
| 50k   | 100k  | 2      | 0.396        | 0.192  | 1          |
| 50k   | 100k  | 5      | 0.329        | 0.155  | 1          |
| 50k   | 100k  | 10     | 0.467        | 0.185  | 1          |

PageRank takes longer as the number of trips (edges) increases, since each iteration processes all edges. Connected Components converges in just 2 iterations at all scales because the 20-station network is always densely connected. Both algorithms remain fast (under 1 second) for all tested combinations.

## Part 3: Partitioning and Replication

In this part, we think about how the database would work in a real distributed system — where data is split across multiple servers (partitioning) and copied to multiple servers for reliability (replication).

### 3.1 Partitioning

Partitioning means splitting the trips collection across multiple servers. We compare two strategies: splitting by `user_id` (each user's trips go to the same server) or by `start_station_id` (trips from the same starting station go to the same server).

## Strategy A: Partition by user\_id

With this strategy, all trips belonging to the same user are stored on the same server. This is very efficient for user-focused queries.

| Query                       | Effect        | Reason                                                                                        |
|-----------------------------|---------------|-----------------------------------------------------------------------------------------------|
| Q1 — trips with user info   | POSITIVE<br>✓ | All trips for a user are on one server — no need to search across servers                     |
| Q2 — user trip statistics   | POSITIVE<br>✓ | All data needed for one user's statistics is in one place                                     |
| Q3 — station trip counts    | NEGATIVE<br>✗ | To count trips per station, we must ask ALL servers and combine results (slow scatter-gather) |
| Q4 — trips with ERROR       | NEUTRAL       | Must scan all servers regardless — events are per-trip and not grouped by user                |
| Graph Q1 — stations by user | POSITIVE<br>✓ | All user trips are local — traversal is fast                                                  |
| Graph Q2 — top stations     | NEGATIVE<br>✗ | Must count trips across all servers to find top stations                                      |
| Spark Q2 — user stats       | POSITIVE<br>✓ | All user trips are on one partition — reduces data shuffling                                  |
| Spark PageRank              | NEGATIVE<br>✗ | Edges (trips) between stations are scattered — requires heavy data movement between servers   |

## Strategy B: Partition by start\_station\_id

With this strategy, all trips starting from the same station are stored on the same server.

| Query                       | Effect        | Reason                                                                                       |
|-----------------------------|---------------|----------------------------------------------------------------------------------------------|
| Q1 — trips with user info   | NEGATIVE<br>✗ | One user's trips are spread across many servers — must search everywhere                     |
| Q2 — user trip statistics   | NEGATIVE<br>✗ | Must collect data from all servers to find all trips for each user                           |
| Q3 — station trip counts    | PARTIAL ✓     | Outgoing trip counts (start) are local; incoming trip counts (end) still need scatter-gather |
| Q4 — trips with ERROR       | NEUTRAL       | No benefit — ERROR events not related to station grouping                                    |
| Graph Q1 — stations by user | NEGATIVE<br>✗ | User trips scattered across all servers                                                      |
| Graph Q2 — top stations     | PARTIAL ✓     | Outgoing counts are local; incoming counts scattered                                         |
| Spark PageRank              | PARTIAL ✓     | Source edges grouped together — reduces some shuffling in each iteration                     |



| Query    | Effect  | Reason                                                                    |
|----------|---------|---------------------------------------------------------------------------|
| Spark CC | NEUTRAL | Only 20 station nodes — partition strategy irrelevant at this small scale |

## Our Recommendation: Partition by user\_id

We recommend partitioning by user\_id for these reasons:

- Most common operations (Q1, Q2, Graph Q1, Spark Q2) are user-focused — they become faster with user partitioning
- The platform is user-centric: users book trips, view their history, and check their statistics — all these are per-user operations
- Q3 (station counts) is slower with user partitioning, but it is an administrative/reporting query that runs infrequently, not a user-facing query
- With 1k to 50k users, the data is spread evenly across partitions — no single user dominates (no hotspots)

## 3.2 Replication

Replication means keeping copies of the data on multiple servers for reliability. We use single-leader asynchronous replication: one primary server handles all writes, and two secondary servers receive copies of the data shortly after. Reads can come from any server.

The key risk: because replication is asynchronous, secondary servers might be slightly behind the primary. If a user just completed a trip and immediately checks their statistics, a secondary server might not have the new trip yet — showing them outdated data.

| Query                         | Risk Level  | What Could Go Wrong                                                                            |
|-------------------------------|-------------|------------------------------------------------------------------------------------------------|
| Q1 — all trips with user info | MEDIUM      | A user might not see their most recently completed trip if it hasn't been replicated yet       |
| Q2 — user trip statistics     | MEDIUM-HIGH | Trip count or average duration could be wrong if new trips are not yet on the secondary server |
| Q3 — station trip counts      | LOW         | Station counts change slowly — being slightly behind by a few seconds is acceptable            |
| Q4 — trips with ERROR         | LOW         | ERROR events are rare and not time-critical — slightly stale data is fine                      |
| Graph Q1 — stations by user   | MEDIUM      | A newly visited station might not appear if the trip wasn't replicated to Neo4j yet            |
| Graph Q2 — top stations       | LOW         | Station rankings change slowly — small lag doesn't affect the top 3                            |
| Spark Q2 — user stats         | MEDIUM-HIGH | Same risk as Q2 — computed averages could be based on incomplete data                          |
| Spark PageRank                | LOW         | PageRank scores change very slowly — minor data lag doesn't change the ranking                 |

| Query    | Risk Level | What Could Go Wrong                                                                       |
|----------|------------|-------------------------------------------------------------------------------------------|
| Spark CC | LOW        | The network is always fully connected — adding or missing a few trips doesn't change this |

## How to Reduce These Risks

- Read-your-writes: After a user completes a trip, send their next query to the primary server (not a secondary), so they always see their own latest data
- Session tokens: Track what each user has written, and make sure secondary servers confirm they have that data before serving reads to that user
- Semi-synchronous replication for critical queries: For Q2 (user statistics), require at least one secondary to confirm it has the data before telling the user the write was successful
- Monotonic reads: Make sure a user always reads from the same server within one session, so they never see data go backward (seeing older data after newer data)

## Conclusion

This project built a complete data management system for a city e-vehicle sharing platform using three different types of databases. Here is what we learned from each:

| Database   | Best For                                                                  | Weakness                                                              |
|------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------|
| PostgreSQL | Joins and structured queries (Q1: 2.09s vs MongoDB 20.36s at large scale) | Query 3: catastrophically slow at scale (957 seconds for 100k trips!) |
| MongoDB    | Flexible schemas and embedded data (Q3: 0.30s vs PostgreSQL 957s)         | Query 1 is 10x slower than PostgreSQL due to \$lookup overhead        |
| Neo4j      | Network traversal — finding connected stations in milliseconds            | Q2 slows significantly when trips are concentrated on few users       |

PySpark is slower than native database engines for single-machine workloads because of startup and data transfer overhead. Spark becomes valuable at truly massive scale — billions of records distributed across many servers in a cluster.

For partitioning, we recommend user-based partitioning because the platform is user-centric and most queries operate per-user. For replication, asynchronous single-leader replication is acceptable for most queries, but user statistics (Q2) need special handling (read-your-writes) to prevent users from seeing outdated data immediately after completing a trip.